The National University of Computer and Emerging Sciences

Islamabad Campus

MLOPS

Assignment # 2

Hamza Asad 22i-1908 DS-C

1. Architecture Overview (Airflow + MLflow)

System Summary:

- End-to-end MLOps pipeline using **Apache Airflow** (workflow orchestration) and **MLflow** (experiment tracking & model registry).
- Runs in **Docker containers** communicating over an internal Docker network.
- Airflow handles task scheduling, dependencies, retries, and data passing (via XCom).
- MLflow records hyperparameters, metrics, and model artifacts; actual ML work is done by Python code in DAG tasks.

Container Architecture:

Container	Role	Notes
PostgreSQL	Airflow metadata DB	Stores task states, XCom values, run histories
Airflow Webserver	UI	localhost:8080
Airflow Scheduler	Orchestrator	Spawns workers, executes DAG tasks
MLflow Tracking Server	Experiment tracking	localhost:5000, metadata in SQLite, artifacts in Docker volume (`--serve-artifacts`)

Airflow ↔ MLflow Communication:

- DAG tasks (e.g., `train_model`) call MLflow Python client → HTTP POST to `http://mlflow:5000`.
- Artifacts are uploaded via MLflow HTTP server (isolated container filesystems).
- Small values (file paths, run IDs, metrics) passed via **XCom**; large objects (models) stored in MLflow.

2. DAG Structure

DAG Name: `mlops_airflow_mlflow_pipeline`

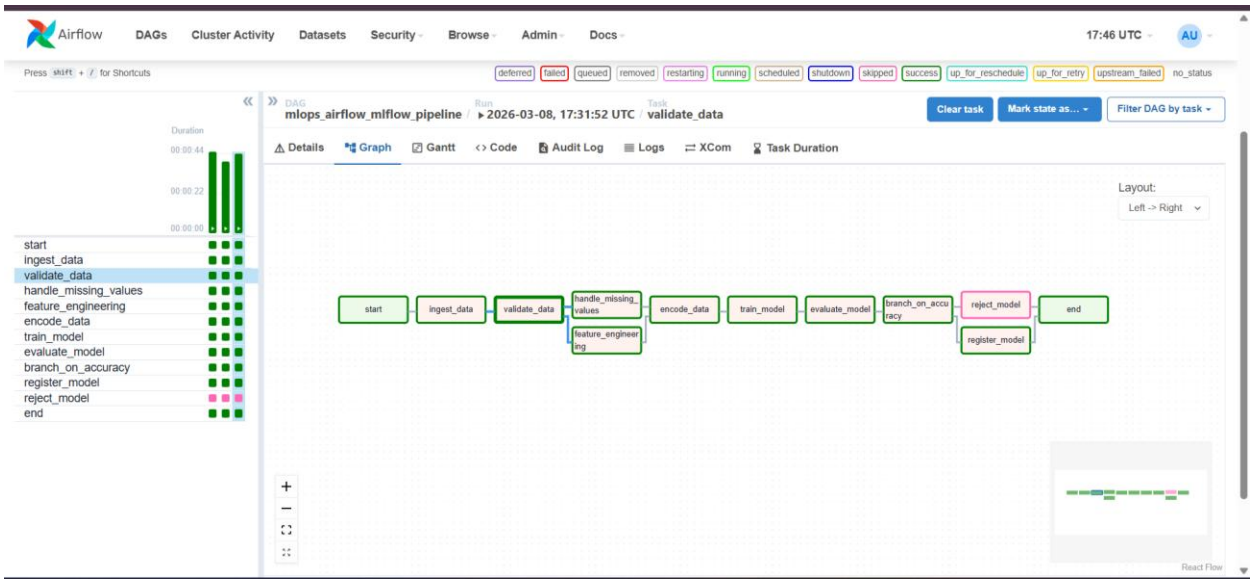
- **Trigger:** Manual (`schedule_interval=None`)
- **Tasks:** 11 tasks, using sequential, parallel (fan-out), fan-in, and conditional branching.

Task Flow:

1. `start` – empty sentinel.
2. `ingest_data` – load `titanic.csv`, log missing values, push path to XCom.
3. `validate_data` – fail-first attempt (demonstrates retries), ensure missing % <30%.

4. Parallel tasks (fan-out) – `handle_missing_values` (fill Age, Embarked, Fare) & `feature_engineering` (FamilySize, IsAlone).
5. `encode_data` – fan-in, merge outputs, encode categorical variables, drop irrelevant columns.
6. `train_model` – split 80/20, MLflow run, log hyperparameters, train Random Forest / Logistic Regression, push `run_id` to XCom.
7. `evaluate_model` – compute accuracy, precision, recall, F1, log metrics in MLflow, push accuracy to XCom.
8. `branch_on_accuracy` – BranchPythonOperator → `"register_model"` if accuracy ≥ 0.80 else `"reject_model"`.
9. `register_model` – register in MLflow Model Registry, stage=Staging.
10. `reject_model` – log rejection reason.
11. `end` – sentinel (`TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS`).

- **No cycles:** strictly acyclic, validated at parse time by Airflow.



3. Experiment Comparison

Runs: Same dataset, 80/20 split, random_state=42.

Run	Model	Accuracy	Precision	Recall	F1
1	RF n=100, d=5	0.8156	0.8750	0.6087	0.7179
2	RF n=200, d=10	0.8156	0.7903	0.7101	0.7481
3	Logistic Regression C=0.5	0.8045	0.7833	0.6812	0.7287

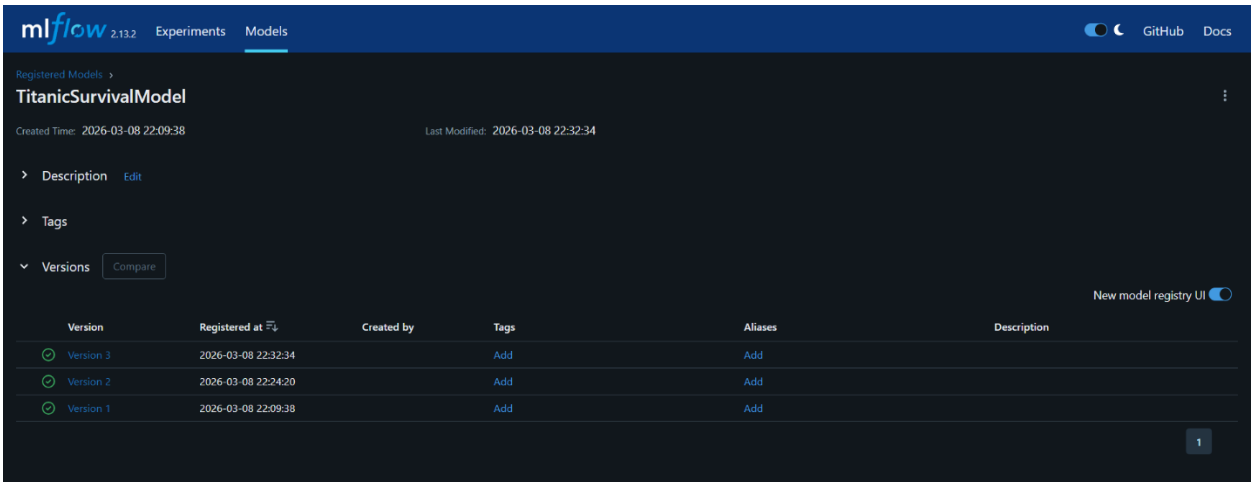
- All three runs exceeded 0.80 accuracy → registered as Versions 1–3 in MLflow.
- Run 2 is the best performing model. While Run 1 and Run 2 share identical accuracy (0.8156), accuracy alone is an insufficient selection criterion. Run 2 achieves a significantly higher F1-score (0.7481 vs 0.7179) and higher recall (0.7101 vs 0.6087).
- In the context of Titanic survival prediction, recall is the more important metric. Recall measures what proportion of actual survivors the model correctly identifies. A low recall means the model misses many real survivors — a false negative. Run 1's recall of 0.6087 means it fails to identify 39% of actual survivors. Run 2 reduces that miss rate substantially.
- Run 1 does have higher precision (0.8750 vs 0.7903), meaning when it predicts survival it is more often correct. However, this comes at the cost of being overly conservative — it predicts survival for fewer passengers overall, missing many real survivors in the process.
- The improvement from Run 1 to Run 2 is explained by the deeper trees (max_depth=10 vs 5) and more estimators (200 vs 100). Deeper trees capture more complex feature interactions such as the combined effect of passenger class, gender, and family size on survival probability. Shallow trees (depth=5) are not expressive enough to learn these multi-feature patterns.
- Logistic Regression in Run 3 underperforms on all metrics, confirming that Titanic survival patterns are inherently non-linear and better modeled by tree-based methods that can learn threshold-based decision rules such as "female AND first class AND age < 40."

dataset_size_test	179	179	179
dataset_size_train	712	712	712
max_depth		10	5
model_type	LogisticRegression	RandomForest	RandomForest
n_estimators		200	100
n_features	9	9	9
random_state	42	42	42
test_size	0.2	0.2	0.2

Metrics

Show diff only

accuracy	0.804	0.816	0.816
f1_score	0.729	0.748	0.718
precision	0.783	0.79	0.875
recall	0.681	0.71	0.609



Summary :

- **Best model:** Run 2 (higher F1 & recall → fewer missed survivors).
- **Insight:** Tree-based models capture non-linear interactions; Logistic Regression underperforms.

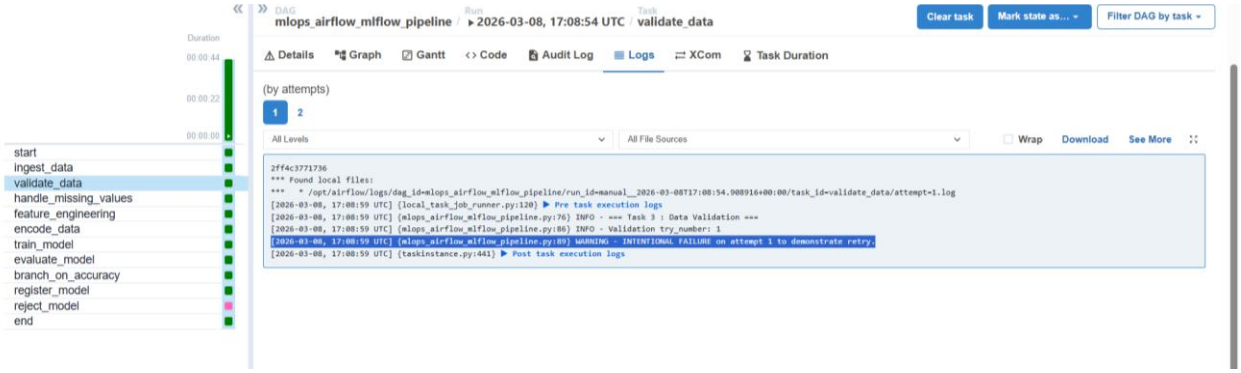
4. Failure & Retry

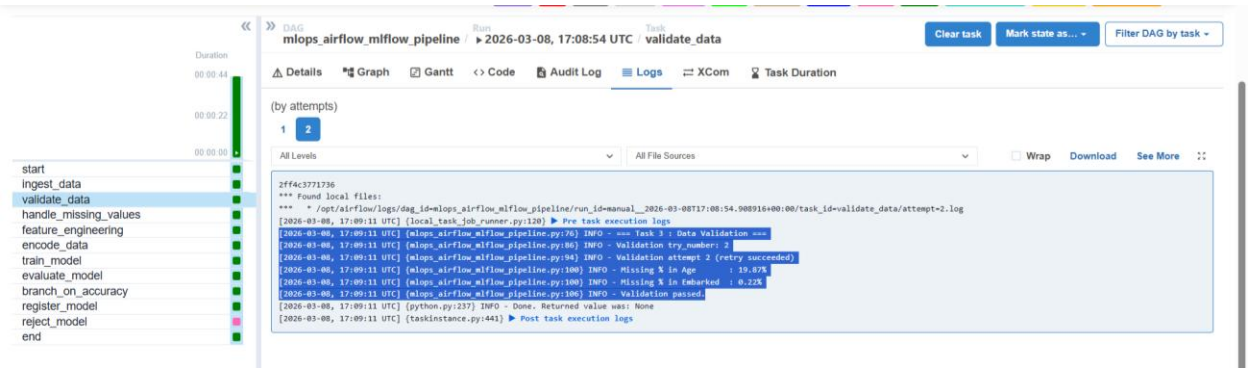
Intentional Failure:

- `validate_data` fails on first attempt to demonstrate retries (`try_number` used, not a Python variable).
- `retries=2`, `retry_delay=10s` → 3 attempts total.

Why `try_number`?

- Airflow spawns a new process per attempt; module-level counters reset every time.
- `try_number` stored in PostgreSQL ensures correct retry tracking.





The validate_data task is configured with retries=2 and retry_delay=timedelta(seconds=10). This gives the task three total attempts — one original execution plus two retries. After the intentional failure on attempt 1, Airflow waits 10 seconds and then re-executes the task. The Airflow UI clearly shows this behavior: the task turns red on attempt 1, turns orange (retrying) after the delay, and then turns green on attempt 2.

All other tasks inherit retries=2 from the DAG's default_args, ensuring the entire pipeline is resilient to transient failures.

5. Production Deployment Considerations

Data Drift:

- Detect with statistical tests (Kolmogorov-Smirnov for numerical, chi-squared for categorical).
- Compare new batches with training baseline stats stored in MLflow.
- Trigger retraining if drift exceeds threshold; log reports with Evidently AI / WhyLogs.

Scaling:

- Replace LocalExecutor with KubernetesExecutor → tasks run in dedicated pods.
- Use **PySpark / Dask** for distributed data processing.
- MLflow backend: SQLite → managed PostgreSQL; artifacts → cloud storage (S3/Blob).

Security:

- Secrets managed via Vault / AWS Secrets Manager, not hardcoded.
- Airflow UI behind HTTPS, RBAC enabled.
- Docker network segmented (MLflow accessible only from Airflow).

CI/CD:

- GitHub Actions: lint, test DAGs, unit & integration tests in lightweight Docker Compose.

- Deploy DAGs via SSH/S3 sync; model promotion triggers serving pipeline.

Monitoring:

- Airflow: `on_failure_callback` → Slack/email, SLA alerts, Prometheus/Grafana dashboards.
- Model: daily inference DAG, track accuracy, auto-trigger retraining if performance drops.